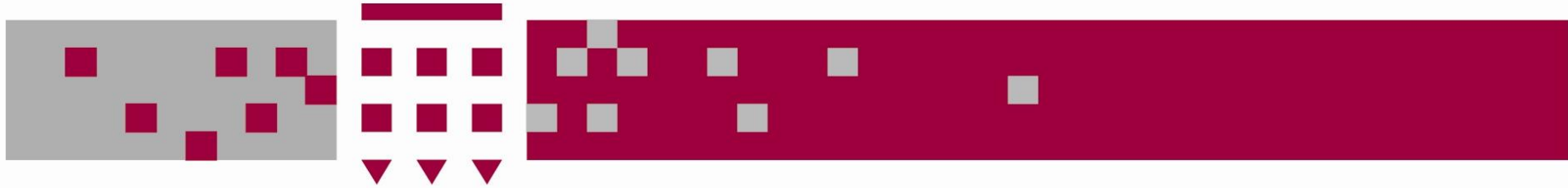


UNIVERSITY OF WESTMINSTER



5COSC019W – Object Oriented Programming Week 8

Dr. Barbara Villarini

b.villarini@westminster.ac.uk



Summary

- Event Handling
- Implementing Event Handler
- Testing - JUnit



What is an Event

- GUI components communicate with the rest of the applications through **events**.
- An **event is an object** that describes a state change in a source
- An event occurs every time the user
 - Types a character or
 - Pushes a mouse button, etc.
- The **source** of an event is the component that causes that event to occur.
- The **listener** of an event is an object that receives the event and processes it appropriately.
- An **event listener** should be registered on the appropriate event source.
- The event causes a call to a method called an **event handler**
- Class **`java.awt.AWTEvent`**

Delegation Event Handling Model

- Three parts
 - Event source
 - GUI component which user interacts with
 - Event object
 - Encapsulates information about event that occurred
 - Event listener
 - Receives event object when notified, then responds
- Programmer must perform two tasks
 1. Register event listener for event source
 2. Implement event-handling method (event handler)

What is an Event Listener?

- An event listener is an **object**
 - It "listens" for events from a specific GUI component (itself an object)
- When an event is generated by the GUI component a method in the listener object is invoked to respond to the event

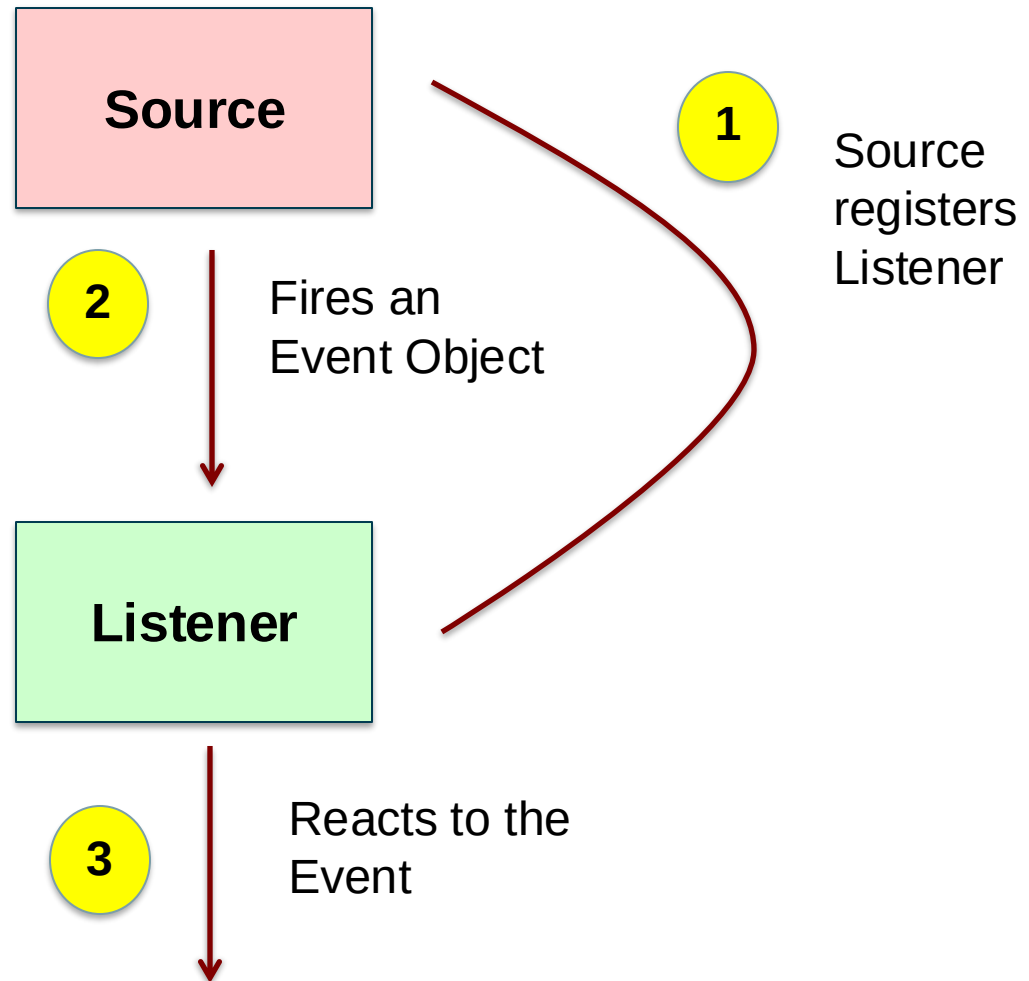


What if there is no event listener?

- When there is no event listener for an event
 - A program can ignore events
 - If there is no listener for an event, the event is just ignored



Recap the Event Handling process



Implement an Event Handler

Every event handler requires three bits of code:

1. Code that specifies that the class either
 1. Implements a listener interface or
 2. Extends a class that implements a listener interface.
2. Code that implements the methods declared in the listener interface.
3. Code that registers an instance of the event handler class as a listener upon one or more components.

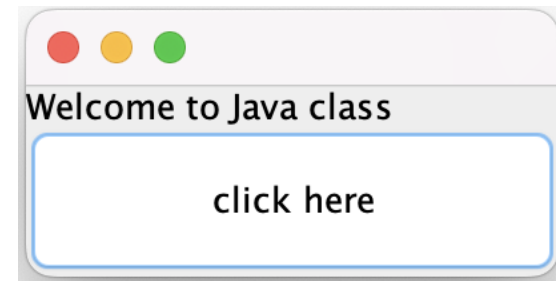
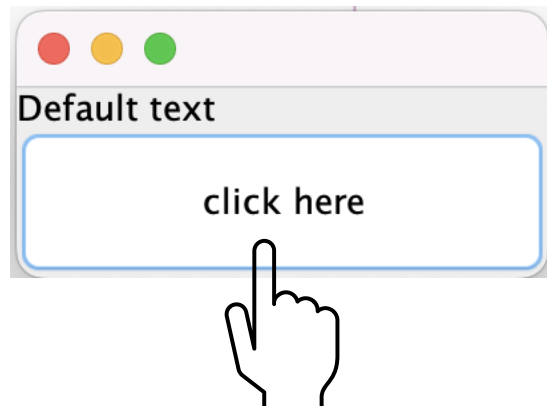
```
public class MyClassListener implements ActionListener {...
```

```
public void actionPerformed(ActionEvent e)  
{ ...//code that reacts to the action... }
```

```
someComponent.addActionListener(instanceOfMyClassListener) ;
```


Example using SimpleGUI

- **JLabel**
 - Single-line area in which text can be displayed
- **JButton**
 - A clickable button





```
public class SimpleGUI extends JFrame{
```

```
    JButton button;  
    JLabel label;
```

} Instance variables – component in SimpleGUI

```
public SimpleGUI(){ ← Constructor
```

```
    button = new JButton("click here");  
    label = new JLabel("Default text");  
    //add components into the frame BorderLayout  
    add(button, BorderLayout.CENTER);  
    add(label, BorderLayout.NORTH);
```

```
    //create an instance of the event handler  
    SimpleEventListener myListener = new SimpleEventListener();  
    button.addActionListener(myListener);
```

} We create an instance of the listener and we register it to the component

```
}
```

```
public class SimpleEventListener implements ActionListener{
```

```
    @Override  
    public void actionPerformed(ActionEvent e) {  
        label.setText("Welcome to Java class");  
    }  
}
```

} We add a listener that implements ActionListener

```
}
```

```
public static void main(String[] args) {
```

```
    SimpleGUI myFrame = new SimpleGUI();  
    myFrame.setVisible(true);  
    myFrame.setSize(200, 100);  
    myFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
}
```

```
}
```



Example using JTextField

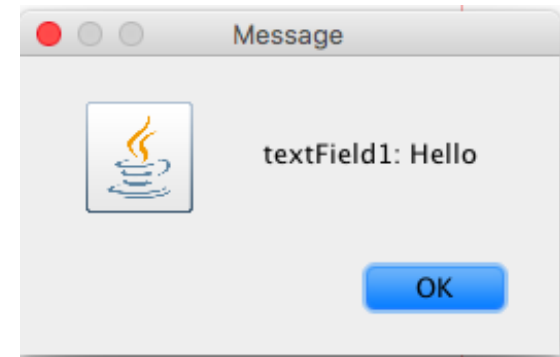
- **JTextField**
 - Single-line area in which user can enter text
- **JPasswordField**
 - Extends **JTextField**
 - Hides characters that user enters



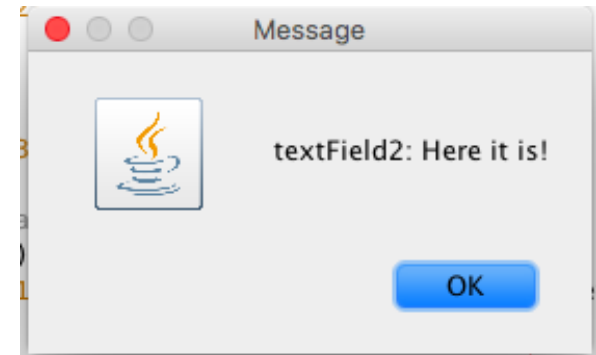
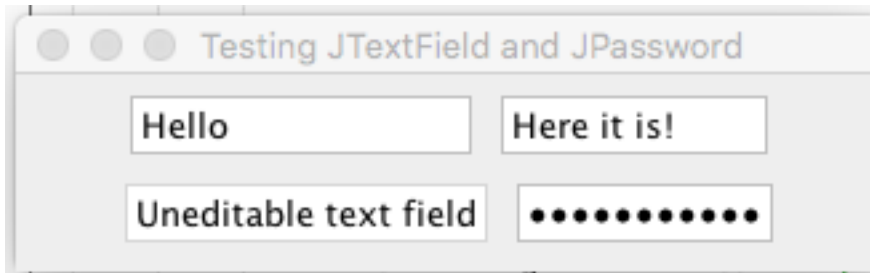
What we visualise:



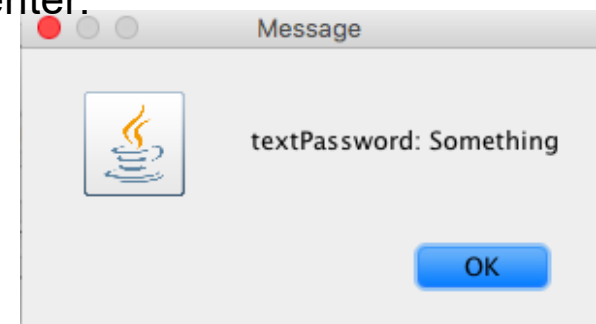
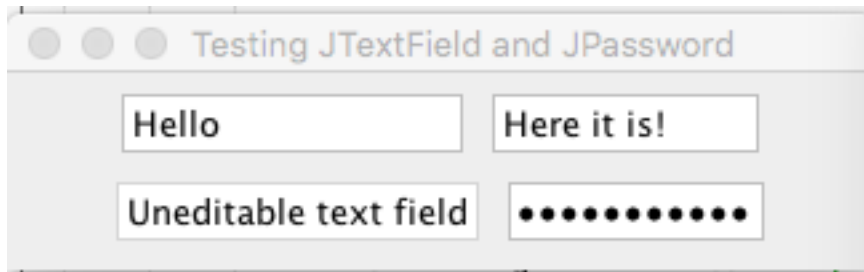
If you enter “Hello!” in the text field 1 and press enter:



If you enter “Here it is!” in the text field 2 and press enter:



If you enter “Something” in the text Password and press enter:





Part (1)

```
package textfieldframe;

import java.awt.FlowLayout;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import javax.swing.JFrame;
import javax.swing.JTextField;
import javax.swing.JPasswordField;
import javax.swing.JOptionPane;

public class TextFieldFrame extends JFrame{
```

Import Swing Packages

← We define our class that extends JFrame

```
    private JTextField textField1;
    private JTextField textField2;
    private JTextField textField3;
    private JPasswordField passwordField;

    // TextFieldFrame constructor adds JTextFields to JFrame
    public TextFieldFrame()
    {
        super("Testing JTextField and JPassword");
        setLayout(new FlowLayout());

        // construct textfield with 10 columns
        textField1 = new JTextField(10);
        add(textField1); // add the textfield to JFrame

        // construct textfield with default text
        textField2 = new JTextField("Enter text here");
        add(textField2); // add the textfield to JFrame
```

Constructor and we define the layout adding the components

Part (2)



```
// construct textfield with default text and 21 columns
textField3 = new JTextField("Uneditable text field");
textField3.setEditable(false);
add(textField3); // add the textfield to JFrame

// construct passwordfield with the default text
passwordField = new JPasswordField("Hidden text");
add(passwordField);

// register event handlers
TextFieldHandler handler = new TextFieldHandler();
textField1.addActionListener(handler);
textField2.addActionListener(handler);
textField3.addActionListener(handler);
passwordField.addActionListener(handler);
} // End TextFieldFrame constructor
```

textField3 will not be editable!

Register event handler to our components:
addActionListener(handler)

```
//private inner class for event handling
private class TextFieldHandler implements ActionListener
{
```

Class for event handling

Override actionPerformed(ActionEvent event)

```
    // process text field events
    public void actionPerformed(ActionEvent event)
    {
        String string = ""; // declare string to display

        // user pressed Enter in JtextField1
        if(event.getSource() == textField1)
            string = String.format("textField1: %s", event.getActionCommand());
```

getSource() is specified by the
EventObject class that ActionEvent is
a child of (via java.awt.AWTEvent)

getActionCommand() gives you a String representing the action command

Part (3)



```
// user pressed Enter in JTextField2
if(event.getSource() == textField2)
    string = String.format("textField2: %s", event.getActionCommand());

// user pressed Enter in JTextField3
if(event.getSource() == textField3)
    string = String.format("textField3: %s", event.getActionCommand());

// user pressed Enter in JTextField passwordField
if(event.getSource() == passwordField)
    string = String.format("textField1: %s", new String(passwordField.getPassword()));

// display the string in a dialog box
JOptionPane.showMessageDialog(null, string);
} // end method action Performed
} // end private inner class

public static void main(String[] args) {

    TextFieldFrame textFieldFrame = new TextFieldFrame();
    textFieldFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    textFieldFrame.setVisible(true);
    textFieldFrame.setSize(325,100);
}
```

Show the string in dialog box

Steps required to set up and event handler

- Create a class for the event handler
- Implement an appropriate event-listener interface
- Register the event handler

Using a Nested Class to Implement an Event Handler

- Top-level classes
 - Not declared within another class
- Nested classes
 - Declared within another class
 - Non-static nested classes are called inner classes
 - Frequently used for event handling
- Note: An inner class is allowed to directly access its top-level class's variables and methods, even if they are private.



Types of Event

Act causing Event	Listener Type
User clicks a button, presses Enter, typing in text field	ActionListener
User closes a frame	WindowListener
Clicking a mouse button, while the cursor is over a component	MouseListener
User moving the mouse over a component	MouseMotionListener
Component becomes visible	ComponentListener
Table or list selection changes	ListSelectionListener

Mouse Event Handling

- Every time an action is performed using a mouse a **MouseEvent** object is created
- Handled by **MouseListeners** and **MouseMotionListeners**
- **MouseListener** combines the two interfaces
- Interface **MouseWheelListener** declares method **mouseWheelMoved** to handle **MouseWheelEvents**

Mouse Events



MouseListener and MouseMotionListener interface methods

*Methods of interface **MouseListener***

public void mousePressed(MouseEvent event)

Called when a mouse button is pressed while the mouse cursor is on a component.

public void mouseClicked(MouseEvent event)

Called when a mouse button is pressed and released while the mouse cursor remains stationary on a component. This event is always preceded by a call to **mousePressed**.

public void mouseReleased(MouseEvent event)

Called when a mouse button is released after being pressed. This event is always preceded by a call to **mousePressed** and one or more calls to **mouseDragged**.

public void mouseEntered(MouseEvent event)

Called when the mouse cursor enters the bounds of a component.

public void mouseExited(MouseEvent event)

Called when the mouse cursor leaves the bounds of a component.

*Methods of interface **MouseMotionListener***

public void mouseDragged(MouseEvent event)

Called when the mouse button is pressed while the mouse cursor is on a component and the mouse is moved while the mouse button remains pressed. This event is always preceded by a call to **mousePressed**. All drag events are sent to the component on which the user began to drag the mouse.

public void mouseMoved(MouseEvent event)

Called when the mouse is moved when the mouse cursor is on a component. All move events are sent to the component over which the mouse is currently positioned.



Example – Part(1)

```
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.event.MouseListener;
import java.awt.event.MouseMotionListener;
import java.awt.event.MouseEvent;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;

public class MouseTrackerFrame extends JFrame{

    private JPanel mousePanel; // panel in which mouse events will occur
    private JLabel statusBar; // label that displays event information

    // MouseTrackerFrame constructor sets up GUI and
    // registers mouse event handlers

    public MouseTrackerFrame()
    {
        super("Demonstarting Mouse Events");

        mousePanel = new JPanel(); // create panel
        mousePanel.setBackground(Color.white);
        add(mousePanel, BorderLayout.CENTER);

        statusBar = new JLabel("Mouse outside JPanel");
        add(statusBar, BorderLayout.SOUTH);
    }
}
```

Create a panel and a label in
MouseTrackerFrame

Part (2)



```
// create and register listener for mouse and mouse motion events
MouseListener handler = new MouseHandler();
mousePanel.addMouseListener(handler);
mousePanel.addMouseMotionListener(handler);
} // end constructor
```

Register listeners for mouse
and mouse motion

```
private class MouseHandler implements MouseListener,
    MouseMotionListener
```

Definition of our class MouseHandler

```
{
    // MouseListener event handlers
    // handle event when mouse released immediatly after press
    public void mouseClicked(MouseEvent event)
    {
        statusBar.setText (String.format("Clicked at [%d, %d]",
            event.getX(), event.getY()));
    }

    // handle event when mouse pressed
    public void mousePressed(MouseEvent event)
    {
        statusBar.setText (String.format("Clicked at [%d, %d]",
            event.getX(), event.getY()));
    }

    //handle event when mouse released after dragging
    public void mouseReleased(MouseEvent event)
    {
        statusBar.setText (String.format("Clicked at [%d, %d]",
            event.getX(), event.getY()));
    }
}
```

Part (3)



```
// handle event when mouse enters area
public void mouseEntered(MouseEvent event)
{
    statusBar.setText (String.format("Clicked at [%d, %d]",
        event.getX(), event.getY()));
    mousePanel.setBackground(Color.green);
}

// handle event when mouse exits area
public void mouseExited(MouseEvent event)
{
    statusBar.setText ("Mouse outside JPanel");
    mousePanel.setBackground(Color.white);
}

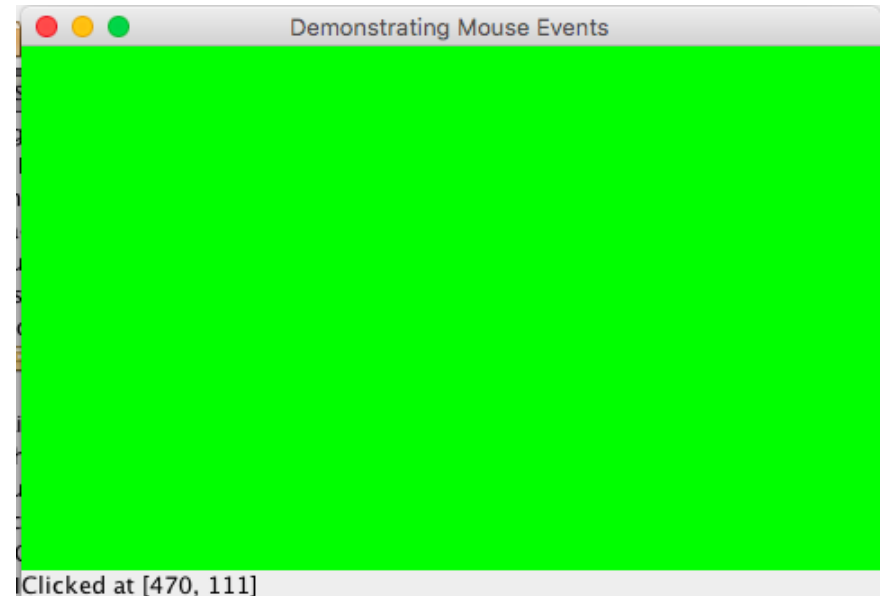
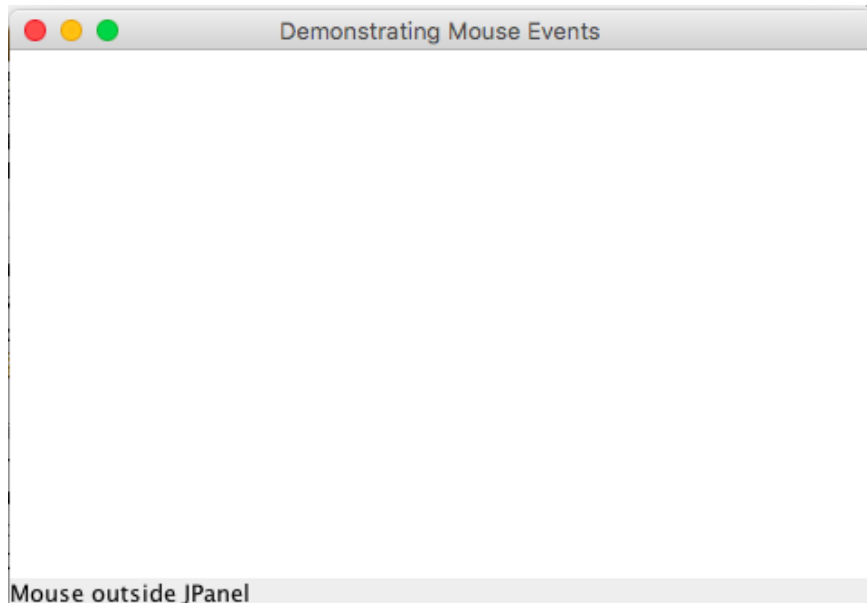
@Override
public void mouseDragged(MouseEvent e) {
    throw new UnsupportedOperationException("Not supported yet."); //To change body of generated
}

@Override
public void mouseMoved(MouseEvent e) {
    throw new UnsupportedOperationException("Not supported yet."); //To change body of generated
}
}
```

Part (4)



```
public static void main(String[] args) {  
  
    MouseTrackerFrame mouseTrackerFrame = new MouseTrackerFrame();  
    mouseTrackerFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    mouseTrackerFrame.setSize(300,100);  
    mouseTrackerFrame.setVisible(true);  
}  
  
}
```



Listener Interface

- Suppose your class directly implements **MouseListener**,
 - Then you must implement all five **MouseListener** methods.
 - Even if you care only about mouse clicks
- Methods for those events you don't care about can have empty bodies.
 - Resulting collection of empty method bodies can make code harder to read and maintain

Adapter Class

- Solution is to use adapter classes
- For example, the **MouseAdapter** class implements the **MouseListener** interface.
- An adapter class implements empty versions of all its interface's methods.
- To use an adapter
 - Create a subclass of it, instead of directly implementing a listener interface.
 - By extending **MouseAdapter**, your class inherits empty definitions of all five of the methods that **MouseListener** contains.

Adapter Classes

- Characteristics of an adapter class
 - Implements interface
 - Provides default implementation of each interface method
 - Used when all methods in interface is not needed

Event-adapter class in <code>java.awt.event</code>	Implements interface
<code>ComponentAdapter</code>	<code>ComponentListener</code>
<code>ContainerAdapter</code>	<code>ContainerListener</code>
<code>FocusAdapter</code>	<code>FocusListener</code>
<code>KeyAdapter</code>	<code>KeyListener</code>
<code>MouseAdapter</code>	<code>MouseListener</code>
<code>MouseMotionAdapter</code>	<code>MouseMotionListener</code>
<code>WindowAdapter</code>	<code>WindowListener</code>



Mouse Adapter in previous code

```
public class MouseTrackerFrame extends JFrame{  
    JPanel panel;  
    JLabel statusBar;  
  
    public MouseTrackerFrame(){  
        panel = new JPanel();  
        panel.setBackground(Color.white);  
  
        statusBar = new JLabel ("Mouse outside the panel");  
  
        add(panel, BorderLayout.CENTER);  
        add(statusBar, BorderLayout.SOUTH);  
  
        MouseHandler myHandler = new MouseHandler();  
        panel.addMouseListener(myHandler);  
        panel.addMouseMotionListener(myHandler);  
    }  
  
    private class MouseHandler extends MouseAdapter {  
  
        @Override  
        public void mouseClicked(MouseEvent e) {  
            statusBar.setText(String.format("Panel Clicked in [%d, %d]", e.getX(), e.getY()));  
            panel.setBackground(Color.green);  
        }  
  
        @Override  
        public void mousePressed(MouseEvent e) {  
            statusBar.setText(String.format("Panel Clicked in [%d, %d]", e.getX(), e.getY()));  
        }  
  
        @Override  
        public void mouseReleased(MouseEvent e) {  
            statusBar.setText(String.format("Panel Clicked in [%d, %d]", e.getX(), e.getY()));  
        }  
  
        @Override  
        public void mouseEntered(MouseEvent e) {  
            statusBar.setText(String.format("Panel Entered in [%d, %d]", e.getX(), e.getY()));  
            panel.setBackground(Color.red);  
        }  
  
        @Override  
        public void mouseExited(MouseEvent e) {  
            statusBar.setText(String.format("Panel Exited"));  
            panel.setBackground(Color.white);  
        }  
  
        @Override  
        public void mouseDragged(MouseEvent e) {  
            statusBar.setText(String.format("Mouse Dragged"));  
        }  
    }  
}
```

If the class extends `MouseAdapter` you do not need to override all the methods!

Clicking a mouse button will always generate which event?

MuseButtonEvent

0%

ActionEvent

0%

MouseEvent

0%

MouseClickedEvent

0%

What two things must your program do to respond to a particular type of event?

Set up an event generator, and set up an event listener.

0%

Create an event listener object for the type of event, and register it with the object that generates those events.

0%

Create a container for the events, and register it with the system.

0%

Create a Frame object and register it with the event listeners

0%

Which interface is commonly used for handling action events, such as button clicks?

MouseListener

0%

ActionListener

0%

EventHandler

0%

EventAdapter

0%

What is the purpose of the getSource() method in the(ActionEvent) class?

To get the event type

0%

to get the source object that generated the event

0%

to get the event timestamp

0%

TESTING



Perfection or Lack of it

- Do your programs work perfectly?
- Are you perfect?



NO!



- No program is perfect.
- Programs will have errors.
- Often see quotes like:

“On average program code has 10 errors per 1000 lines...”



Testing

- Testing will help to find bugs, by actually running the code.
- Testing *cannot* show your program will always work properly – Remember that also the testing can have some bugs!
- But it can remove sufficient bugs to make your program “good enough”.
- Testing allows you to gain confidence in your code.



Black and white box testing

What is the difference between black- and white-box testing?

- **black-box** (procedural) **test**: Written without knowledge of how the system under test is implemented.
 - focuses on input/output of each component or call
- **white-box** (structural) **test**: Written with knowledge of the implementation of the code under test.
 - focuses on internal states of objects and code
 - focuses on trying to cover all code paths/statements
 - requires internal knowledge of the component to craft input
 - example: knowing that the internal data structure for a spreadsheet uses 256 rows/columns, test with 255 or 257



Black box testing and Tests plan

- black-box is based on requirements and functionality, not code
- tester may have actually seen the code before ("gray box")
 - but doesn't look at it while constructing the tests
- often done from the end user perspective
- emphasis on parameters, inputs/outputs (and their validity)

Test Scenario Description	Test Case Description	Test Steps	Pre-Conditions	Expected Result	Test Data	Status
Verify the login Security	Check user login (for an existing account) with invalid email and password.	Enter invalid email and password while initially logging in to the app, then click on the login button.	User must have the app open. user must enter incorrect details.	A text box to pop up asking the user to re-enter a correct/registered email or password.	email: invalidEmail password: invalidpassword	Passed
Verify the login Security	Check user login with valid Data.	Enter valid email and password then click on login button to gain access to the app.	User must have the app open. user must enter correct details.	The application to log into the account as they press the login button.	email: Validname password: validpassword	Passed
Verify the login Security	Check what happens when the login button is pressed without any information.	Press the login button without entering any data into the text fields for email and password.	User must have the app open.	Text to appear and the email and password text field to highlight in red letting the user know the field is empty.	email: password:	Passed
Verify the login Security	Check how many times an incorrect password has been entered.	Enter an invalid password three times.	User must have an existing account User must have the app open. User must enter incorrect password.	After the user enters the password 3 times, a message box will appear asking the user to contact the teacher or the admin to get their account back	password: invalidpassword(x3)	Failed
Check app Functionality	Check if the application lets the user select the context of their conversation.	Display a screen with all the possible contexts of a conversation in the form of buttons.	User must have an existing account User must be logged in. User must have the app open.	Display the conversation for the selected context.		Passed
Check app Functionality and usability	If the user clicks on a tricky word, does the translation box show up?	User clicks on the highlighted word (tricky word)	User must have an existing account User must have logged in. User must have the app open. User must have selected the language, language level, context and subcontext.	Display a pop up box, with tricky words translated in the chosen language.		Failed
Check app Functionality	When the user completes a level, does the progress bar increase.	user completes a conversation at their selected level and then user clicks on the finish level button.	User must have an existing account. User must have logged in. User must have the app open.	The progress bar increases by a set amount (depending on the number of levels completed).		Passed
Check app Functionality	When the user signs in, is the progress from the previous session saved?	User enters their login details and then clicks on the login button	User must have an existing account User must have logged in User must have started on a task previously User must have the app open.	The levels completed from a previous session should be greyed out and progress bar should be the same from the last session.		Failed
		The user has to complete a conversation at their chosen	User must have an existing account. User must have logged in.		Pick a language and then	



White Box testing

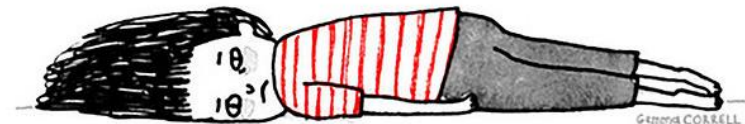
- "static testing"
 - code walkthroughs, inspections, code reviews
 - static analysis tools
 - CheckStyle <http://checkstyle.sourceforge.net/>
 - FindBugs <http://findbugs.sourceforge.net/>
 - code complexity analysis tools
 - PMD, CheckStyle, etc.
- **Unit testing**: a piece of code written by a developer that executes a specific functionality in the code to be tested and asserts a certain behaviour or state.
- The percentage of code which is tested by unit tests is typically called **test coverage**.



Testing and Proof

- To prove something we must show: $\forall x \cdot P(x)$
- This implies we have to explore every possible state a program can be in.
- But how can we explore all the possible state???
- Let's consider a method that **compute the sqrt of a number...**
- To test if it works correctly we should try all the possible float number, which are $2^{64} \cong 10^{19}!!!!$
- That means we can take ages!!!

NOPE.



Running tests

- Construct the test dataset : select **representatives** from each of the class
- Create a test harness — a program to call sqrt with the elements of the test dataset.
- Run the program and compare the results with what was expected (which you need to work out some other way!).
- **Need an automated approach.**
 - Write a test program that reads data from a data structure or file.
 - Get the program to run tests and check the results



JUnit

- JUnit is a simple, free and open source framework to write repeatable unit tests using Java.
- It is a regression-testing framework, which allows the developer to write code faster with high quality.
- JUnit tests increase the stability of the software.
- The main philosophy behind this testing framework is to make coding and testing move hand in hand.
- JUnit provides annotations, assertions, and test runners to facilitate the creation and execution of unit tests.



Key concepts in JUnit

1. Annotations:

JUnit uses annotations to identify methods that specify a test. Common annotations include `@Test`, `@Before`, `@After`, `@BeforeClass`, and `@AfterClass`.

1. **@Test**: Marks a method as a test method. JUnit will run methods annotated with `@Test` as test cases.
2. **@BeforeEach**: Runs before each test method. It is often used to set up the test environment (e.g., initialise variables).
3. **@AfterEach**: Runs after each test method. It's used to clean up the test environment (e.g., release resources).
4. **@BeforeClass**: Runs once before any of the test methods in the class. Typically used for one-time setup tasks that apply to all tests.
5. **@AfterClass**: Runs once after all the test methods in the class have been run. It's used for cleanup that should only happen once after all tests finish.

2. Assertions:

Assertions are used to verify that certain conditions are met during the execution of a test. If an assertion fails, the test will fail.

Common Assertion Methods:

- **assertEquals**(expected, actual): Checks if two values are equal.
- **assertTrue**(condition): Checks if a condition is true.
- **assertFalse**(condition): Checks if a condition is false.
- **assertNotNull**(object): Checks if an object is not null.



Key concepts in JUnit

3. Test Case:

A test case is essentially a class that contains one or more test methods. Each test method, annotated with `@Test`, contains code that verifies a particular feature or function of the code being tested.

Structure: Within each test method, you'll generally set up test data, perform some operation, and then use assertions to validate the outcome.

4. Test Suites:

A test suite is a way to group multiple test cases or even other test suites together, allowing related tests to be organised and run together.

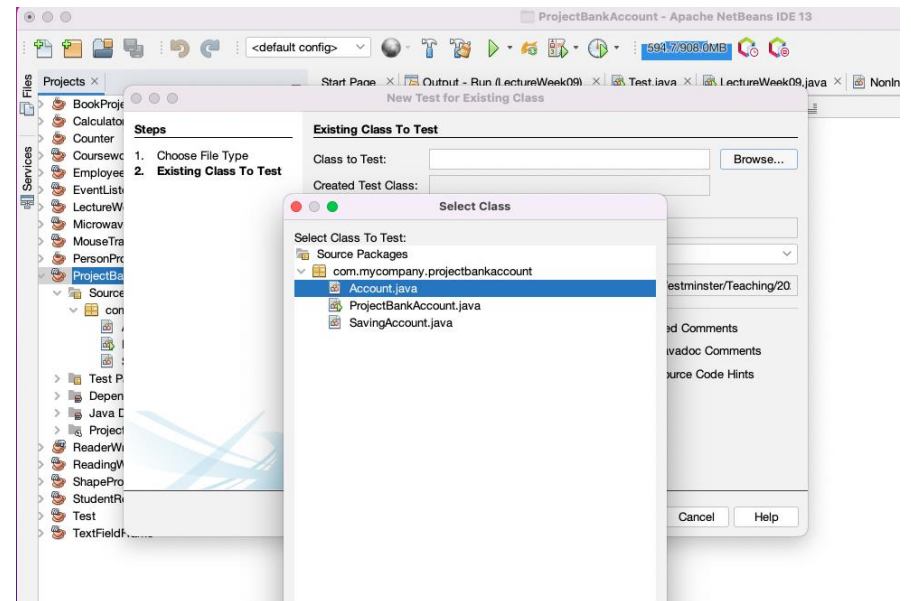
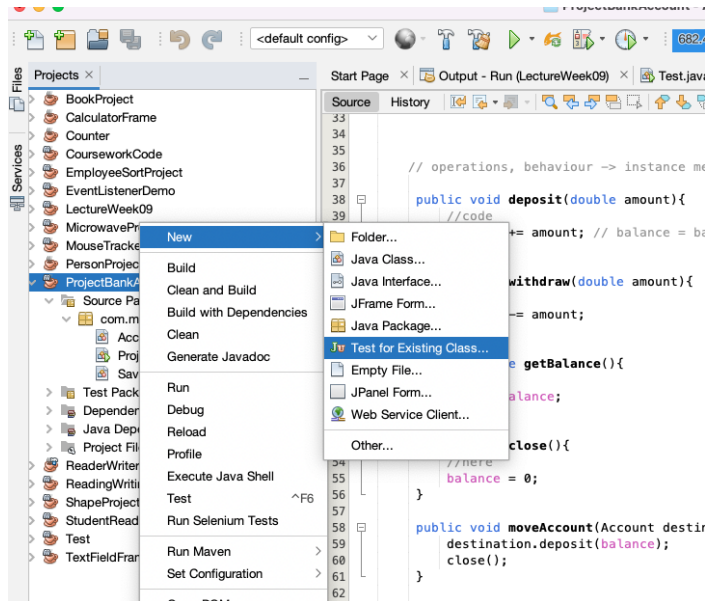
Usage: You can create a test suite to run all tests for a specific module, feature, or the entire project at once. In JUnit 5, `@Suite` can be used to define a test suite, and individual test classes are included in the suite

A case study- Create a Junit Test for deposit method

```
public class BankAccount {  
  
    private double balance;  
    private String accountNumber;  
  
    public BankAccount(double initialBalance){  
        balance = initialBalance;  
  
    }  
    public void deposit(double amount){  
        //code  
        balance += amount;  
    }  
    public void withdraw(double amount){  
        //code  
        balance -= amount;  
    }  
    public double getBalance() {  
        //code  
        return balance;  
    }  
    public void close() {  
        //here  
        balance = 0;  
    }  
}
```



How to create a test:





How to write a test case

```
public class AccountTest {  
  
    public BankAccount testAccount;  
  
    public AccountTest() {  
    }  
  
    @BeforeEach  
    public void setUp() {  
        testAccount = new BankAccount(50);  
    }  
  
    @Test  
    public void testDeposit() {  
  
        testAccount.deposit(50);  
  
        BankAccount accExpected = new BankAccount(100);  
  
        assertEquals(accExpected.getBalance(), testAccount.getBalance());  
    }  
}
```

@BeforeEach - Runs before each test method

@Test - @Test annotation marks the method as a test.

assertEquals checks if the result of the add method is equal to the expected value.



Assert

- `assertEquals(expected, actual)`
- `assertEquals(message, expected, actual)`
- `assertEquals(expected, actual, delta)`
- `assertEquals(message, expected, actual, delta)`
- `assertFalse(condition)`
- `assertFalse(message, condition)`
- `Assert(Not)Null(object)`
- `Assert(Not)Null(message, object)`
- `Assert(Not)Same(expected, actual)`
- `Assert(Not)Same(message, expected, actual)`
- `assertTrue(condition)`
- `assertTrue(message, condition)`

Well-structured Assertions

```
public class BankAccountTest {  
    @Test  
    public void depositTest1() {  
        BankAccount account = new BankAccount(50);  
        account.deposit(50);  
  
        assertEquals(100, account.getBalance(), 0.001);    }
```

 The expected value has to be in the left

```
    public void depositTest2() {  
        BankAccount account = new BankAccount(50);  
        account.deposit(-50);  
  
        assertEquals(50, account.getBalance(), 0.001);    }
```


Tips for testing

- You cannot test every possible input, parameter value, etc.
 - So you must think of a limited set of tests likely to expose bugs.
- Think about boundary cases
 - positive; zero; negative numbers
 - right at the edge of an array or collection's size
- Think about empty cases and error cases
 - 0, -1, null; an empty list or array
- test behavior in combination
 - maybe `add` usually works, but fails after you call `remove`
 - make multiple calls; maybe `size` fails the second time only